

Advanced Algorithms: Homework 4 Solutions

Due on Feb. 7, 2024 at 11:59pm EST

Professor Dana Randall Spring 2024

As stated in the syllabus, unauthorized use
of previous semester course materials is
strictly prohibited in this course.

Exercise 1

For some problem P , two online algorithms A_1 and A_2 are given, with a competitive ratio of 2 and 3, respectively. Design a randomized online algorithm with competitive ratio $9/4$.

Solution

At the very start, choose A_1 with probability $3/4$ and A_2 with probability $1/4$. Then the expected output of our algorithm is at most $\leq 3/4 \cdot 2OPT + 1/4 \cdot 3OPT = 9/4 \cdot OPT$.

Exercise 2

We consider a version of the game Memory with just one player. As in the usual game, let n pairs of cards lie on the table, face down. In each move of the game, the player can turn two cards, one at a time. If they are identical, they are removed, otherwise they are turned face down again and put back in their spots. The goal of the player is to remove all cards with the least number of moves. Even after the successful turning of a pair, any further turning is considered a move. We assume that the player can remember all cards that have been turned at any time. The optimal strategy needs n moves to remove all $2n$ cards.

- Design an online strategy that takes at most $2n$ moves. i.e. that is 2-competitive
- Design an online strategy that takes at most $2n - 1$ moves
- Show that any deterministic strategy can be forced to take at least $2n - 1$ moves.
- What changes about the competitive ratio if the player gets a free move after removing a pair?

Solution

- Flip all $2n$ cards over in n moves to learn where all the cards are. Now, remove every pair in another n moves, for a total of $2n$ moves. As OPT could have removed all cards in n moves (if it knew where every card was ahead of time), this algorithm is $2n/n = 2$ -competitive.
- Flip $2n - 2$ of the cards over in $n - 1$ moves. There are two cards whose identities we haven't revealed. There are two cases:
 - All of the $2n - 2$ cards that you flipped pair up evenly. Then you know that the two hidden cards also form a pair. So, pick all the cards up in another n moves. This costs a total of $2n - 1$ moves: $n - 1$ moves to flip over all but 2 cards, and then n moves to remove all pairs.
 - All of the $2n - 2$ cards do not match up evenly. This means that the two hidden cards do NOT form a pair. Do the following: reveal ONE of the hidden cards. Its pair is something you already uncovered, so select that card to get a match. Do the same with the other hidden card. Now, match up all the remaining $2n - 4$ cards, all known, which must all pair evenly. This also costs a total of $2n - 1$ moves: $n - 1$ moves to flip over all but 2 cards, and then n moves to remove all pairs, starting with the two cards that weren't revealed already.
- Given any deterministic strategy, we adversarially pick a choice of cards that will require the strategy to take $2n - 1$ moves. Essentially, whenever the deterministic strategy makes a choice that causes a new card to be revealed, we as the adversary will choose the worst possible card.

We use the following idea when planning our strategy: if ALG matches cards that it had flipped over earlier, then it is wasting moves. ALG would prefer to create a match on the fly while uncovering at least one of those cards. By creating a match on the fly, we mean ALG flips over a card whose pair it already knows, so for its second card of the move it can flip over that card and create the pairing.

Thus it uses the same move both to uncover a new card and to remove a pair. So, we as the adversary will place cards to minimize the number of times ALG creates a match on the fly with unknown cards.

Also, if ALG has revealed two cards that match, it doesn't matter when ALG picks them up. ALG can pick them up immediately or save it for the end. We save all these pick ups for the end in our analysis.

We also note that ALG has no reason to flip an already revealed card and then an unrevealed card - ALG can switch the order and only gain information.

We outline our strategy as the adversary below:

- On the first move, ALG reveals one card, and then reveals another card. We, as the adversary, ensure that they are different so ALG doesn't get a match, say (without loss of generality) these are 1 and 2.
- On the next move, ALG reveals another card - we can't let it be 1 or 2, so let it be 3. ALG doesn't have a match, so it should reveal a new card, which we as the adversary pick to be 2. Now, there is a matched 2 known to us, and an unmatched 1 and 3.
- On the i th turn, repeat this idea: there is an unmatched 1 and i on the field, and both cards labeled $2, 3, \dots, i-1$ have been revealed. So, we introduce $i+1$ as the first new card, and i as the second new card. This can continue for $n-1$ steps.

Now we have forced, in $n-1$ steps, for ALG to not get any matches on the fly. Now, ALG must pick up the remaining n pairs, and will be forced to take at least $2n-1$ steps.

- (d) If we get a free move after a match, then OPT could pick up every card in only one move. Our ALG still needs at least $n-1$ steps to discover all the cards, then it can pick up all the cards in one move, for a total of n steps. Thus the competitive ratio is n .

Exercise 3

For the Paging Problem, let ALG be any marking algorithm as presented in the lecture with a cache of size k , and let OPT be an optimal off-line-algorithm with a cache of size $h \leq k$. Prove that ALG is $\frac{k}{k-h+1}$ -competitive.

Hint: Using the strategy from class where $h = k$, consider a decomposition of the request sequence σ into phases that are maximal subsequences with k distinct pages requested.

Solution

As the hint says, we'll consider any input sequence σ of page requests, and divide it into sequential phases, where each phase is a maximal substring containing no more than k distinct pages. By splitting into phases in this way, we know that our marking algorithm ALG will fault at most k times per phase, from when we studied marking algorithms.

However, OPT only has a cache of size h . We want to show that OPT will fault at least $k-h+1$ times in each phase. The pigeonhole principle, used naively, will only get us $k-h$ faults per phase, so we need to be a little more careful.

OPT starts the phase with h elements in its cache, call this set of elements $CACHE$. We will see k items in this phase. The first element x in the current phase was not in the previous phase. Thus there are two cases:

- x is not among the h elements in the $CACHE$. Then, at most $h-1$ elements in the $CACHE$ will be seen in the current phase. The pigeonhole principle then guarantees $k-h+1$ faults.

- x is among the h elements in the CACHE. Since x was not in the previous phase, x must have been in the cache since at least two phases earlier. Thus, it is as if the previous phase only had a cache of size $h - 1$ to work with, since x was occupying slot. This forces the previous phase to have at least one “extra” fault, outside of the accounting of that phase’s own forced faults. Thus, OPT will have at least $k - h$ faults in this phase, and cause one extra fault in the previous phase.

In either case, summing the total faults across all phases will show that OPT fails a total of $k - h + 1$ times the number of phases, plus or minus a constant amount of accounting for the first and last phases. Thus, we have competitive ratio $\frac{k}{k-h+1}$.